

Министерство науки и высшего образования Российской Федерации

федеральное государственное автономное
образовательное учреждение высшего образования
«Самарский национальный исследовательский университет
имени академика С.П. Королева»

Институт информатики, математики и электроники

Факультет информатики

Кафедра технической кибернетики

Отчет по курсу

Дисциплина: «Технологии сетевого программирования»

Выполнили: Иванова Д. М.,

Мещеряков С.Е

Группа: 6301-010302D

Самара, 2025

СОДЕРЖАНИЕ

1 Концепция приложения	3
1.1 Назначение приложения	3
1.2 Состав приложения	3
1.3 Описание функциональных частей	3
1.3.1 Серверная часть	3
1.3.2 Клиентская часть	3
1.3.3 База данных	5
1.3.4 Нейросеть	5
1.4 Функционирование приложения	5
2 Работа приложения	6
2.1 Регистрация и авторизация	6
2.2 Генерация изображений	8
3 Ссылка на репозиторий GitHub	12
Приложение А	13
Приложение Б	21
Приложение В	23
Приложение Г	44

1 Концепция приложения

1.1 Назначение приложения

Приложение предназначено для генерации изображений по текстовому описанию.

1.2 Состав приложения

Приложение состоит из 4 частей:

- 1) Серверная часть приложения.
- 2) Клиентская часть приложения.
- 3) База данных.
- 4) Нейросеть.

1.3 Описание функциональных частей

1.3.1 Серверная часть

Серверная часть приложения написана на языке программирования Python с использованием фреймворка Django.

Также в ходе работы использовался Django REST Framework для написания методов API.

1.3.2 Клиентская часть

Клиентская часть реализована на React JS + css. В папке styles в css файлах хранятся стили для светлой и темной (пока нету) тем и для кастомных модулей – кнопок и форм ввода. Реализация формы ввода представлена на рисунке 1. Пример css файла представлен на рисунке 2. Страницы приложения хранятся в папке pages: страница регистрации, входа, главная страница, страница смены пароля. В папке router задаются пути и соответствующие им страницы. Пример приведен на рисунке 3.

```
import React from "react";
import style from '../../styles/MyInput.module.css';

const MyInput = React.forwardRef((props, ref) => {
  return (
    <input ref={ref} {...props} className={style.MyInput}/>
  );
});

export default MyInput;
```

Рисунок 1 – JSX файл с кодом для кастомной формы ввода

```
.MyInput {
  border-radius: 15px;
  width: 90%;
  height: 47px;
  margin-top: 5%;
  margin-top: 20px;
  border: none;
  outline: none;
}

input {
  font-family: 'Montserrat', 'semi-bold';
  font-size: 0.75rem;
  text-indent: 20px;
}

::-webkit-input-placeholder {
  /* WebKit/Blink browsers */
  font-family: 'Montserrat', 'semi-bold';
  font-size: 0.75rem;
}

::-moz-placeholder {
  /* Firefox 19+ */
  font-family: 'Montserrat', 'semi-bold';
  font-size: 0.75rem;
}

:-ms-input-placeholder {
  /* Internet Explorer 10-11 */
  font-family: 'Montserrat', 'semi-bold';
  font-size: 0.75rem;
}

:-moz-placeholder {
  /* Mozilla Firefox 4 to 18 */
  font-family: 'Montserrat', 'semi-bold';
  font-size: 0.75rem;
}

::placeholder {
  font-family: 'Montserrat', 'semi-bold';
  font-size: 0.75rem;
}
```

Рисунок 2 – CSS файл со стилями для формы ввода

```

//import ChangeCode from "../pages/ChangeCode.jsx";
import ChangePass from "../pages/ChangePass.jsx";
//import CodeLight from "../pages/Light/CodeLight.jsx";
//import Links from "../pages/Links.jsx";
import Login from "../pages/Login.jsx";
import Main from "../pages/Main.jsx";
//import OurTeam from "../pages/OurTeam.jsx";
import Registration from "../pages/Registration.jsx";

export const privateRoutes = [
]

export const publicRoutes = [
  {path: '/login', component: Login, exact: true},
  {path: '/registration', component: Registration, exact: true},
  //{path: '/code', component: CodeLight, exact: true},
  {path: '/main', component: Main, exact: true},
  //{path: '/change-code', component: ChangeCode, exact: true},
  {path: '/change-pass', component: ChangePass, exact: true},
  //{path: '/ourteam', component: OurTeam, exact: true},
  //{path: '/links', component: Links, exact: true}
]

```

Рисунок 3 – JS файл со стилями для формы ввода

1.3.3 База данных

В качестве СУБД была выбрана PostgreSQL. Для работы с базой использовались ORM модели, предлагаемые фреймворком Django.

1.3.4 Нейросеть

Нейросеть написана с помощью PyTorch. Для коммуникации с нейросетью используется единственный API метод, реализованный с помощью фреймворка FastAPI.

1.4 Функционирование приложения

Приложение собрано через docker-compose.yml файл. Образы функциональных частей в сумме весят 11,5 гигабайт.

2 Работа приложения

2.1 Регистрация и авторизация

Для получения доступа к основному функционалу приложения необходим вход в учётную запись. Если учётной записи не существует, то её можно зарегистрировать.

Авторизация выполнена с помощью стандартных инструментов фреймворка Django. В качестве модели пользователя используется своя модель пользователя, наследованная от `AbstractUser`. Для авторизации пользователя используются встроенные методы `TokenObtainPairView` и `TokenRefreshView`.

```
class User(AbstractUser):  
    id = models.AutoField(primary_key=True)  
    email = models.EmailField(unique=True)  
  
    def __str__(self):  
        return self.userName
```

Рисунок 4 – Модель пользователя

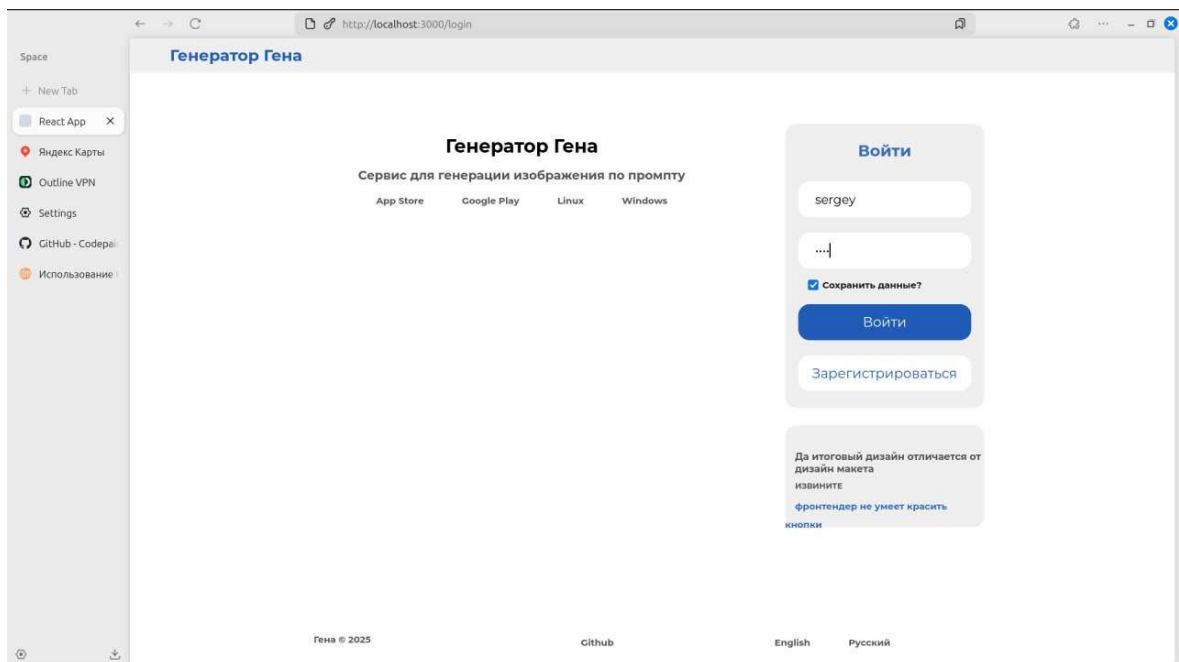


Рисунок 5 – Страница входа

Регистрация пользователя происходит через сериализатор, получающий значения полей и регистрирующий пользователя в базе данных.

```
class RegisterUserView(APIView):
    permission_classes = [AllowAny]
    def post(self, request):
        serializer = UserSerializer(data=request.data)
        try:
            if serializer.is_valid():
                serializer.save()
                return Response(serializer.data, status=status.HTTP_201_CREATED)
            else:
                return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)
        except Exception as e:
            logger.error(f"Ошибка регистрации пользователя: {str(e)}")
            return Response({"error": "Internal Server Error"}, status=status.HTTP_500_INTERNAL_SERVER_ERROR)
```

Рисунок 6 – Регистрация пользователя

```
class UserSerializer(serializers.ModelSerializer):
    class Meta:
        model = User
        fields = ['id', 'username', 'email', 'password']
        extra_kwargs = {'password': {'write_only': True}}

    def create(self, validated_data):
        user = User.objects.create_user(**validated_data)
        return user

    def update(self, instance, validated_data):
        if 'password' in validated_data:
            instance.set_password(validated_data.pop('password'))
        return super().update(instance, validated_data)
```

Рисунок 7 – Сериализатор для пользователя

Для регистрации пользователю необходимо указать логин, пароль и адрес электронной почты. Проверка пароля осуществляется неявно внутренними средствами фреймворка.

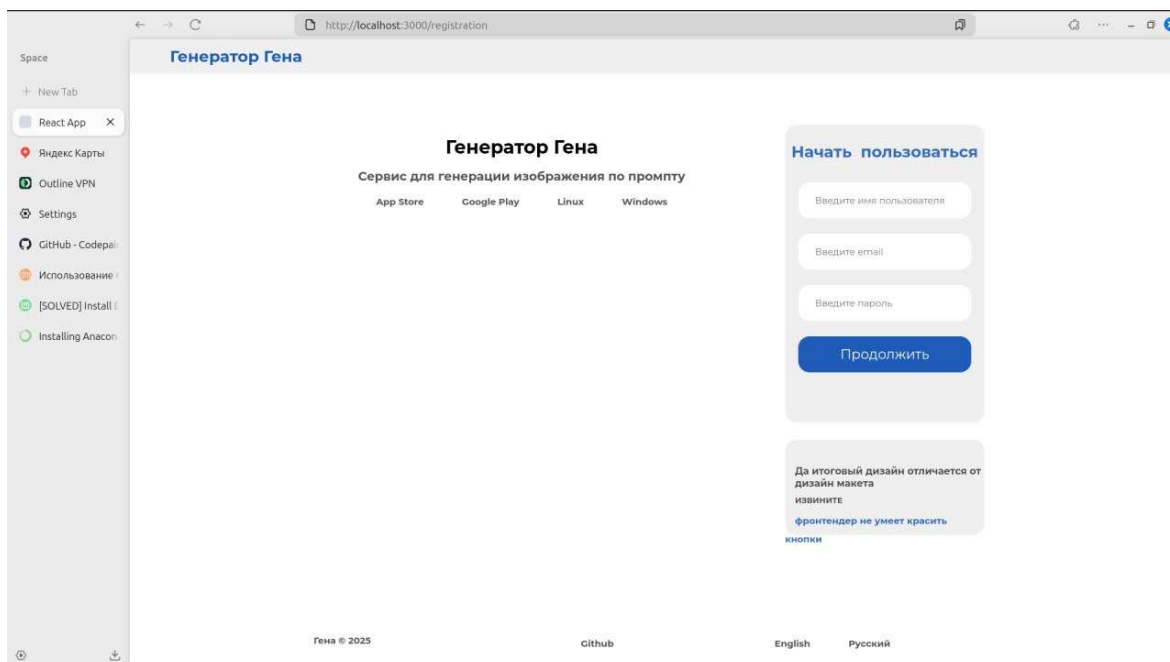


Рисунок 8 – Страница регистрации

2.2 Генерация изображений

На главной странице сайта доступно поле ввода для генерации изображений. Текст должен быть на английском языке.

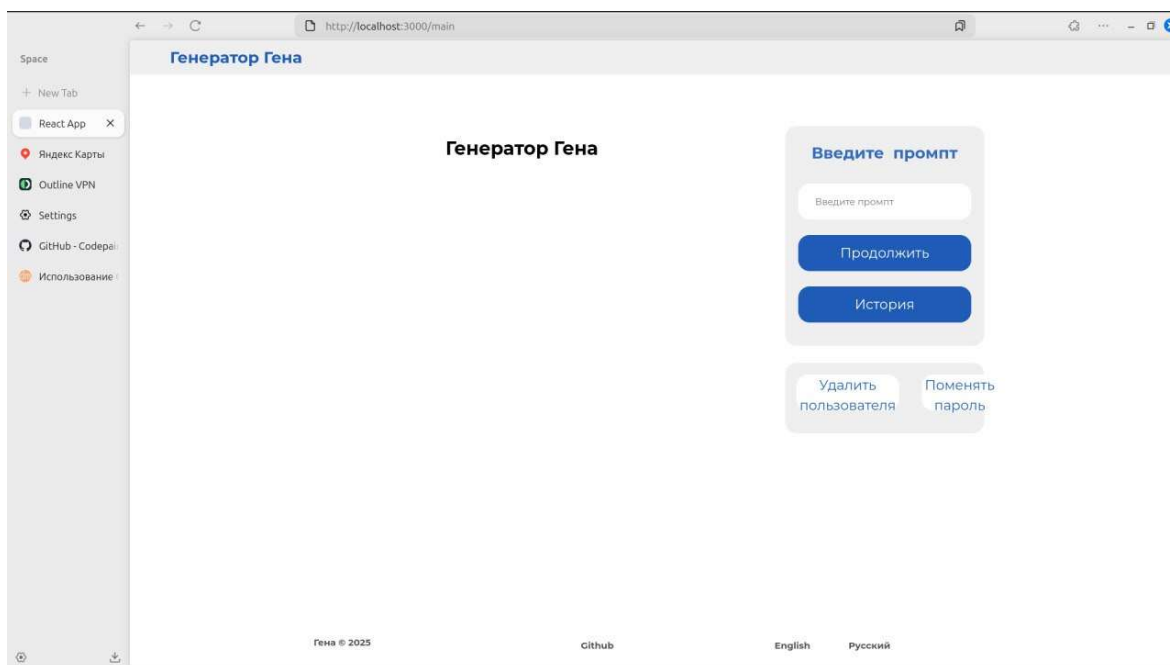


Рисунок 9 – Главная страница

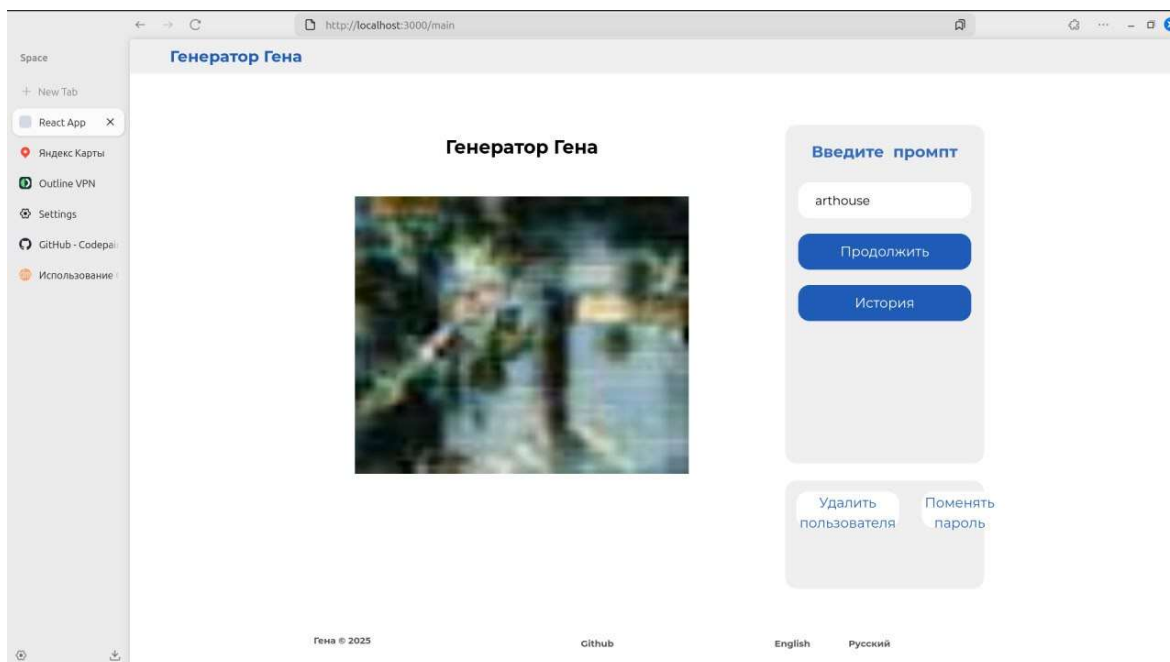


Рисунок 10 – Сгенерированное изображение

В процессе выполнения задания тестировалось несколько архитектур модели. Из-за ограничений ресурсов на данный момент используется самая простая, генерирующая изображения с разрешением 64x64. Основной проблемой при обучении моделей был mode collapse – генератор либо находил несколько изображений, с которыми можно обманывать дискриминатор, либо дискриминатор оказывался слишком сильным и генератор сваливался в генерацию случайного шума даже после 60 эпох обучения. Код сделан таким образом, что модели можно легко заменять корректировкой одной строчки с загрузкой весов модели и использования соответствующего кода для архитектуры модели. Модель вынесена как отдельное FastAPI приложение.

```

from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from gena import Gena
import traceback

app = FastAPI()
model = Gena()

class GenerateRequest(BaseModel):
    prompt: str

@app.post("/generate/")
def generate_image(data: GenerateRequest):
    try:
        result = model.generate_from_text(data.prompt)
        return {"images": result}
        #print(f"Received prompt: {data.prompt}")
        # временный фейковый ответ
        #return {"images": ["base64_stub_image_data"]}
    except Exception as e:
        print("Error while generating image:")
        traceback.print_exc()
        raise HTTPException(status_code=500, detail=str(e))

```

Рисунок 11 – FastAPI приложение

```

class Generator(nn.Module):
    def __init__(self, noise_dim=100, text_dim=512, out_channels=3):
        super().__init__()
        self.projection = nn.Linear(text_dim, 256)
        self.fc = nn.Linear(noise_dim + 256, 512 * 4 * 4)

        self.net = nn.Sequential(
            nn.BatchNorm2d(512),
            nn.ConvTranspose2d(512, 256, 4, 2, 1), # 8x8
            nn.BatchNorm2d(256),
            nn.ReLU(True),
            nn.ConvTranspose2d(256, 128, 4, 2, 1), # 16x16
            nn.BatchNorm2d(128),
            nn.ReLU(True),
            nn.ConvTranspose2d(128, 64, 4, 2, 1), # 32x32
            nn.BatchNorm2d(64),
            nn.ReLU(True),
            nn.ConvTranspose2d(64, out_channels, 4, 2, 1), # 64x64
            nn.Tanh()
        )

    def forward(self, noise, text_emb):
        projected_text = self.projection(text_emb)
        x = torch.cat([noise, projected_text], dim=1)
        x = self.fc(x).view(-1, 512, 4, 4)
        return self.net(x)

```

Рисунок 12 – Архитектура генератора

3 Ссылка на репозиторий GitHub

<https://github.com/Codepairs/generatorgena>

ПРИЛОЖЕНИЕ А

В приложении приведен исходный код сервиса и базы данных.

Методы:

```
from rest_framework import status
from rest_framework.response import Response
from rest_framework.views import APIView

from rest_framework.permissions import IsAuthenticated, AllowAny
from rest_framework_simplejwt.authentication import JWTAuthentication
from rest_framework_simplejwt.tokens import RefreshToken

from gena_database_app.models import User, UsageHistory, ImageModel
from .serializers import UserSerializer, ChangePasswordSerializer, UsageHistorySerializer,
ImageModelSerializer, \
                                GenaGenerateSerializer

from django.conf import settings
import logging
import requests

import base64
from django.http import HttpResponse
from io import BytesIO
from PIL import Image

logger = logging.getLogger(__name__)

class RegisterUserView(APIView):
    permission_classes = [AllowAny]
    def post(self, request):
        serializer = UserSerializer(data=request.data)
        try:
            if serializer.is_valid():
                serializer.save()
                return Response(serializer.data, status=status.HTTP_201_CREATED)
            else:
                return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)
        except Exception as e:
            logger.error(f"Ошибка регистрации пользователя: {str(e)}")
            return Response({"error": "Internal Server Error"},
status=status.HTTP_500_INTERNAL_SERVER_ERROR)

class GetUserView(APIView):
    permission_classes = [IsAuthenticated]
    def get(self, request):
        try:
```

```

        user_id = request.query_params.get('user_id')
        if request.user.id != int(user_id):
            return Response({"message": "Доступ к данным другого пользователя запрещен"},
status=status.HTTP_403_FORBIDDEN)
        user = User.objects.get(id=user_id)
        serializer = UserSerializer(user) # many=False, так как получаем одного пользователя
        return Response(serializer.data, status=status.HTTP_200_OK)
    except User.DoesNotExist:
        return Response({"message": "Пользователь не найден"},
status=status.HTTP_404_NOT_FOUND)

class UpdateUserView(APIView):
    permission_classes = [IsAuthenticated]
    def put(self, request):
        try:
            user_id = request.query_params.get('user_id')
            if request.user.id != int(user_id):
                return Response({"message": "Доступ к данным другого пользователя запрещен"},
status=status.HTTP_403_FORBIDDEN)
            user = User.objects.get(id=user_id)
        except User.DoesNotExist:
            return Response(status=status.HTTP_404_NOT_FOUND)

        serializer = UserSerializer(user, data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data)
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

class DeleteUserView(APIView):
    permission_classes = [IsAuthenticated]
    def delete(self, request):
        try:
            user_id = request.query_params.get('user_id')
            if request.user.id != int(user_id):
                return Response({"message": "Доступ к данным другого пользователя запрещен"},
status=status.HTTP_403_FORBIDDEN)
            user = User.objects.get(id=user_id)
        except User.DoesNotExist:
            return Response(status=status.HTTP_404_NOT_FOUND)

        user.delete()
        return Response(status=status.HTTP_204_NO_CONTENT)

class GetUserHistoryView(APIView):
    permission_classes = [IsAuthenticated]
    def get(self, request):
        try:
            user_id = request.query_params.get('user_id')
            if request.user.id != int(user_id):

```

```

        return Response({"message": "Доступ к данным другого пользователя запрещен"},
status=status.HTTP_403_FORBIDDEN)
        user = User.objects.get(id=user_id)
        history = UsageHistory.objects.filter(userID=user)
        serializer = UsageHistorySerializer(history, many=True)
        return Response(serializer.data, status=status.HTTP_200_OK)
    except User.DoesNotExist:
        return Response({"message": "Пользователь не найден"},
status=status.HTTP_404_NOT_FOUND)

class GetRequestView(APIView):
    permission_classes = [IsAuthenticated]
    def get(self, request, request_id):
        try:
            user_id = request.query_params.get('user_id')
            if request.user.id != int(user_id):
                return Response({"message": "Доступ к данным другого пользователя запрещен"},
status=status.HTTP_403_FORBIDDEN)
            user = User.objects.get(id=user_id)
            operation_id = request.data.get("operation_id")
            history = UsageHistory.objects.filter(userID=user, operationID=operation_id)
            serializer = UsageHistorySerializer(history, many=True)
            return Response(serializer.data, status=status.HTTP_200_OK)
        except User.DoesNotExist:
            return Response({"message": "Пользователь не найден"},
status=status.HTTP_404_NOT_FOUND)

class LogoutView(APIView):
    permission_classes = [IsAuthenticated]
    def post(self, request):
        try:
            refresh_token = request.data.get("refresh_token")
            token = RefreshToken(refresh_token)
            token.blacklist()
            return Response({"message": "Выход выполнен"}, status=status.HTTP_200_OK)
        except Exception as e:
            return Response({"error": str(e)}, status=status.HTTP_400_BAD_REQUEST)

class ChangePasswordView(APIView):
    permission_classes = [IsAuthenticated]
    def put(self, request):
        serializer = ChangePasswordSerializer(data=request.data, context={'request': request})
        if serializer.is_valid():
            serializer.update(request.user, serializer.validated_data)
            return Response({"message": "Пароль успешно изменён"}, status=status.HTTP_200_OK)
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

##### GENA

class GenerateImage(APIView):
    permission_classes = [IsAuthenticated]

```

```

def post(self, request):
    #user_id
    #prompt

    #try:
    #    serializer = GenaSerializer(data=request.data)
    #    serializer.is_valid(raise_exception=True)
    #    client = serializer.save()
    #except requests.exceptions.RequestException as e:
    #    return Response({'error': 'Ошибка сервера'},
status=status.HTTP_500_INTERNAL_SERVER_ERROR)

    try:
        generate_serializer = GenaGenerateSerializer(data=request.data)
        generate_serializer.is_valid(raise_exception=True)
        data = generate_serializer.validated_data

        if request.user.id != data['user_id']:
            return Response({"message": "Доступ к данным другого пользователя запрещен"},
status=status.HTTP_403_FORBIDDEN)

        response = requests.post(
            "http://ml_service:5000/generate/",
            json={"prompt": data['prompt']}
        )
        if response.status_code != 200:
            return Response({'error': 'Ошибка генерации'},
status=status.HTTP_500_INTERNAL_SERVER_ERROR)

        result = response.json()["images"]

        operation_info = {
            'userID': data['user_id'],
            'prompt': data['prompt'],
            'status': 'created'
        }

        history_serializer = UsageHistorySerializer(data=operation_info)
        history_serializer.is_valid(raise_exception=True)
        history_instance = history_serializer.save()
    except requests.exceptions.RequestException:
        return Response({'error': 'Ошибка сервера'},
status=status.HTTP_500_INTERNAL_SERVER_ERROR)

    try:
        image_info = {
            'image_base64': result[0]
        }

        imgae_serializer = ImageModelSerializer(data=image_info)
        imgae_serializer.is_valid(raise_exception=True)
        image_instance = imgae_serializer.save()

```



```

        history_instance.imageID = image_instance
        history_instance.status = 'generated'
        history_instance.save()

        response_serializer = UsageHistorySerializer(history_instance)

        return Response(response_serializer.data, status=status.HTTP_201_CREATED)
    except requests.exceptions.RequestException:
        return Response({'error': 'Ошибка сервера'},
            status=status.HTTP_500_INTERNAL_SERVER_ERROR)

#####

# получение изображения файлом
class GetImageView(APIView):
    permission_classes = [IsAuthenticated]
    def get(self, request):
        try:
            operation_id = request.query_params.get("operation_id")
            user_id = request.query_params.get("user_id")

            if request.user.id != int(user_id):
                return Response({"message": "Доступ к данным другого пользователя запрещен"},
                    status=status.HTTP_403_FORBIDDEN)
            operation = UsageHistory.objects.get(operationID=operation_id, userID = user_id)
            image = operation.imageID
            raw_image = image.image_base64

            # Декодируем base64
            image_data = base64.b64decode(raw_image)
            img = Image.open(BytesIO(image_data))

            # Сохраняем в поток в формате JPG
            img_io = BytesIO()
            img.save(img_io, format='JPEG')
            img_io.seek(0)

            # Возвращаем файл пользователю
            response = HttpResponse(img_io, content_type='image/jpeg')
            response['Content-Disposition'] = f'attachment; filename="{image.imageID}.jpg"'
            return response
        except ImageModel.DoesNotExist:
            return Response({"message": "Изображение не найдено"},
                status=status.HTTP_404_NOT_FOUND)
        except Exception as e:
            return Response({"message": str(e)}, status=status.HTTP_500_INTERNAL_SERVER_ERROR)

```

Сериализаторы:

```
from rest_framework import serializers
from gena_database_app.models import User, ImageModel, UsageHistory

class UserSerializer(serializers.ModelSerializer):
    class Meta:
        model = User
        fields = ['id', 'username', 'email', 'password']
        extra_kwargs = {'password': {'write_only': True}}

    def create(self, validated_data):
        user = User.objects.create_user(**validated_data)
        return user

    def update(self, instance, validated_data):
        if 'password' in validated_data:
            instance.set_password(validated_data.pop('password'))
        return super().update(instance, validated_data)

class ChangePasswordSerializer(serializers.Serializer):
    old_password = serializers.CharField(write_only=True)
    new_password = serializers.CharField(write_only=True)

    def validate_old_password(self, value):
        user = self.context['request'].user
        if not user.check_password(value):
            raise serializers.ValidationError("Старый пароль неверен.")
        return value

    def update(self, instance, validated_data):
        instance.set_password(validated_data['new_password'])
        instance.save()
        return instance

class ImageModelSerializer(serializers.ModelSerializer):
    class Meta:
        model = ImageModel
        fields = ['imageID', 'image_base64', 'createdAt', 'rating']

    def create(self, validated_data):
        image = ImageModel.objects.create(
            image_base64=validated_data.get('image_base64', ''),
            rating=validated_data.get('rating', 0.0)
        )
        return image

    def update(self, instance, validated_data):
        instance.image_base64 = validated_data.get('image_base64', instance.image_base64)
        instance.rating = validated_data.get('rating', instance.rating)
```

```

instance.save()
return instance

```

```

class UsageHistorySerializer(serializers.ModelSerializer):
    class Meta:
        model = UsageHistory
        fields = ['operationID', 'userID', 'imageID', 'prompt', 'createdAt', 'updatedAt', 'status']

    def create(self, validated_data):
        usage = UsageHistory.objects.create(
            userID=validated_data.get('userID', ''),
            prompt=validated_data.get('prompt', ''),
            status=validated_data.get('status', 'created'),
            imageID=None
        )
        return usage

    def update(self, instance, validated_data):
        instance.prompt = validated_data.get('prompt', instance.prompt)
        instance.status = validated_data.get('status', instance.status)
        instance.imageID = validated_data.get('imageID', instance.imageID)
        instance.save()
        return instance

```

```

class GenaGenerateSerializer(serializers.Serializer):
    user_id = serializers.IntegerField(required=True)
    prompt = serializers.CharField(required=True)
    images = serializers.IntegerField(default=1, required=False)
    width = serializers.IntegerField(default=1024, required=False)
    height = serializers.IntegerField(default=1024, required=False)

    def validate_images(self, value):
        if value < 1:
            raise serializers.ValidationError("Количество изображений должно быть не меньше 1.")
        return value

    def validate_width(self, value):
        if value <= 0:
            raise serializers.ValidationError("Ширина должна быть положительным числом.")
        return value

    def validate_height(self, value):
        if value <= 0:
            raise serializers.ValidationError("Высота должна быть положительным числом.")
        return value

```

ORM модели:

```

from django.db import models
from django.contrib.auth.hashers import make_password, check_password

from django.contrib.auth.models import AbstractUser

#class User(models.Model):
#    userID = models.AutoField(primary_key=True)
#    email = models.EmailField(unique=True)
#    password = models.CharField(max_length=128)
#    userName = models.CharField(max_length=50)

#    def set_password(self, raw_password):
#        self.password = make_password(raw_password)

#    def check_password(self, raw_password):
#        return check_password(raw_password, self.password)

#    def __str__(self):
#        return self.userName

class User(AbstractUser):
    id = models.AutoField(primary_key=True)
    email = models.EmailField(unique=True)

    def __str__(self):
        return self.userName

class ImageModel(models.Model):
    imageID = models.AutoField(primary_key=True)
    image_base64 = models.TextField()
    createdAt = models.DateTimeField(auto_now_add=True)
    rating = models.FloatField(default=0.0)

    def __str__(self):
        return f"Image {self.imageID} - {self.image_base64}"

class UsageHistory(models.Model):
    operationID = models.AutoField(primary_key=True)
    userID = models.ForeignKey(User, on_delete=models.CASCADE)
    imageID = models.ForeignKey(ImageModel, on_delete=models.SET_NULL, null=True)
    prompt = models.TextField()
    createdAt = models.DateTimeField(auto_now_add=True)
    updatedAt = models.DateTimeField(auto_now=True)
    status = models.CharField(max_length=20)

    def __str__(self):
        return f"Operation {self.operationID} - User {self.userID} - Image {self.imageID}"

```

ПРИЛОЖЕНИЕ Б

В приложении приведен исходный код нейросети.

Код модели:

```
import torch
from transformers import CLIPTokenizer, CLIPTextModel
import json
import base64
from io import BytesIO
from torchvision.utils import make_grid
from torchvision.transforms.functional import to_pil_image
from torch import nn

class Generator(nn.Module):
    def __init__(self, noise_dim=100, text_dim=512, out_channels=3):
        super().__init__()
        self.projection = nn.Linear(text_dim, 256)
        self.fc = nn.Linear(noise_dim + 256, 512 * 4 * 4)

        self.net = nn.Sequential(
            nn.BatchNorm2d(512),
            nn.ConvTranspose2d(512, 256, 4, 2, 1), # 8x8
            nn.BatchNorm2d(256),
            nn.ReLU(True),
            nn.ConvTranspose2d(256, 128, 4, 2, 1), # 16x16
            nn.BatchNorm2d(128),
            nn.ReLU(True),
            nn.ConvTranspose2d(128, 64, 4, 2, 1), # 32x32
            nn.BatchNorm2d(64),
            nn.ReLU(True),
            nn.ConvTranspose2d(64, out_channels, 4, 2, 1), # 64x64
            nn.Tanh()
        )

    def forward(self, noise, text_emb):
        projected_text = self.projection(text_emb)
        x = torch.cat([noise, projected_text], dim=1)
        x = self.fc(x).view(-1, 512, 4, 4)
        return self.net(x)

class Gena:
    def __init__(self):
        self.tokenizer = CLIPTokenizer.from_pretrained("openai/clip-vit-base-patch32")
        self.text_encoder = CLIPTextModel.from_pretrained("openai/clip-vit-base-patch32")
        self.generator = Generator()

    def encode_text(self, captions):
        inputs = self.tokenizer(captions, padding=True, return_tensors="pt", truncation=True)
        with torch.no_grad():
            text_features = self.text_encoder(**inputs).last_hidden_state[:, 0, :]
```

```

        return text_features

    def to_base64(self, generated):
        base64_images = []

        for img_tensor in generated:
            img = to_pil_image(img_tensor.clamp(0, 1)) # тензор в [0,1] -> PIL
            buffered = BytesIO()
            img.save(buffered, format="PNG")
            img_str = base64.b64encode(buffered.getvalue()).decode("utf-8")
            base64_images.append(img_str)

        return base64_images

    def generate_from_text(self, prompt, num_images=1):
        self.generator.load_state_dict(torch.load('./Generator_70.pt',
map_location=torch.device('cpu'))))
        self.generator.eval()
        with torch.no_grad():
            text_emb = self.encode_text([prompt]*num_images).to('cpu')
            noise = torch.randn(num_images, 100).to('cpu')
            generated = self.generator(noise, text_emb).cpu()

        generated = self.to_base64(generated)
    return generated

```

Код приложения:

```

from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from gena import Gena
import traceback

app = FastAPI()
model = Gena()

class GenerateRequest(BaseModel):
    prompt: str

@app.post("/generate/")
def generate_image(data: GenerateRequest):
    try:
        result = model.generate_from_text(data.prompt)
        return {"images": result}
        #print(f"Received prompt: {data.prompt}")
        # временный фейковый ответ
        #return {"images": ["base64_stub_image_data"]}
    except Exception as e:
        print("Error while generating image:")
        traceback.print_exc()
        raise HTTPException(status_code=500, detail=str(e))

```

ПРИЛОЖЕНИЕ В

В приложении приведен исходный код клиентской части.

Страницы:

```
import React, {useState, Suspense } from "react";
import style from "../styles/Light/Main.module.css";
import MyButton from "../UI/components/buttons/MyButton";
import MyInput from "../UI/components/input/MyInput";
import "bootstrap/dist/css/bootstrap.min.css";
import { useTranslation } from "react-i18next";
import { useHistory } from "react-router-dom";

const ChangePassLight = () => {
  const [oldPassword, setOldPassword] = useState('');
  const [newPassword, setNewPassword] = useState('');

  // Обработчик изменения для input'ов
  const handlePasswordChange = (event) => {
    const { placeholder, value } = event.target;

    if (placeholder === 'Введите старый пароль') {
      setOldPassword(value);
    } else if (placeholder === 'Введите новый пароль') {
      setNewPassword(value);
    }
  };

  async function changePassword() {
    // Переменные для смены пароля
    const token = localStorage.getItem("accessToken");

    // URL API для смены пароля
    const url = 'http://localhost:8000/api/users/change-password/';

    // Тело запроса в формате JSON
    const body = {
      old_password: oldPassword,
      new_password: newPassword,
    };

    try {
      const response = await fetch(url, {
        method: 'PUT', // Метод PUT как в серверной функции
        headers: {
          'Content-Type': 'application/json', // Тип контента
          'Authorization': `Bearer ${token}`, // Токен авторизации
        },
        body: JSON.stringify(body), // Преобразуем тело в JSON
      });

      if (response.ok) {
```

```

    const data = await response.json();
    console.log('Успех:', data.message); // Ожидаем сообщение "Пароль успешно изменён"
  } else {
    const errorData = await response.json();
    console.error('Ошибка:', errorData);
  }
} catch (error) {
  console.error('Ошибка сети или запроса:', error);
}
}

// переход между страницами
const historyHook = useHistory();
const returnToLogin = () => {
  historyHook.push("/login");
};
const returnOurTeam = () => {
  historyHook.push("/ourteam");
};
const { t, i18n } = useTranslation("translation");
return (
  <div>
    <Suspense fallback={<div>Loading...</div>}></Suspense>
    /** контейнер страницы */
    <div className={style.LoginPage}>
      /** заголовок*/
      <div className={style.Header}>
        <a onClick={returnToLogin} className={style.HeaderLink}>
          Генератор Гена
        </a>
      </div>
      <div className="container-fluid">
        <div className={style.MainPage}>
          <div className="row">
            <div className={"col-lg-2"}></div>
            <div className={`col-lg-5 text-center`}>
              <div className={style.Info}>
                <h2>Генератор Гена</h2>
              </div>
            </div>
          </div>
          <div className={`col-lg-3`}>
            <div className={style.MainForm}>
              <form className={style.form}>
                <h1>Обновить пароль</h1>
                <MyInput
                  style={{ margin: "0px", marginTop: "8%" }}
                  type="password"
                  placeholder="Введите старый пароль"
                  onChange={handlePasswordChange}
                />
                <MyInput

```



```

        style={{ margin: "0px", marginTop: "8%" }}
        type="password"
        placeholder="Введите новый пароль"
        onChange={handlePasswordChange}
      />
      <MyButton
        style={{
          backgroundColor: "#1F5CB6",
          color: "ffffff",
          margin: "0px",
          marginTop: "8%",
        }}
        onClick={changePassword}>
        Продолжить
      </MyButton>
    </form>
  </div>
  <div className={style.QR}>

    </div>
  </div>
  {/** ссылки внизу страницы */}
  <div className={style.Bottom}>
    <h4>Гена © 2025</h4>
    <a
      className={style.text}
      onClick={returnOurTeam}
      style={{ marginRight: "15%" }}
    >
      Github
    </a>
    <a
      onClick={() => i18n.changeLanguage("en")}
      className={style.text}
    >
      English
    </a>
    <a
      onClick={() => i18n.changeLanguage("ru")}
      className={style.text}
      style={{ marginRight: "10%" }}
    >
      Русский
    </a>
  </div>
</div>
</div>
</div>
</div>
);

```

```

};
export default ChangePassLight;

import React, { useContext, useState } from "react";
import { AuthContext } from "../context";
import style from "../styles/Light/Login.module.css";
import MyButton from "../UI/components/buttons/MyButton";
import MyInput from "../UI/components/input/MyInput";
import 'bootstrap/dist/css/bootstrap.min.css';
import { useTranslation } from 'react-i18next';
import { useHistory } from 'react-router-dom';

const LoginLight = () => {
  // состояния для авторизации (об это я подумаю позже)
  const { isAuth, setIsAuth } = useContext(AuthContext);
  // переменные
  const [loginData, setLoginData] = useState({ name: '', password: '' });
  // не помню зачем делала, но пусть пока будут
  const [submitted, setSubmitted] = useState(false);
  // для чтения логина из поля ввода
  const handleUsernameChange = (event) => {
    const name = event.target.value;
    setLoginData({ ...loginData, name: name });
  };
  // для чтения пароля из поля ввода
  const handlePasswordChange = (event) => {
    const password = event.target.value;
    setLoginData({ ...loginData, password });
  };
  const handleRedirectToMain = () => {
    history.push('/main');
  };
  const [isRemember, setRemember] = useState()

  const login = async (event) => {
    event.preventDefault();
    setSubmitted(true);

    const url = 'http://localhost:8000/api/users/login/';

    const data = {
      username: loginData.name,
      password: loginData.password
    };
    console.log('AAAAAAAAAAAA')
    try {
      const response = await fetch(url, {
        method: 'POST',
        headers: {
          'Content-Type': 'application/json'
        },
      },

```

```

        body: JSON.stringify(data)
    });

    if (!response.ok) {
        console.error('Ошибка при выполнении запроса:');
        return
    }

    const result = await response.json();

    localStorage.setItem('accessToken', result.access);
    localStorage.setItem('refreshToken', result.refresh);
    handleRedirectToMain();
    console.log(result)
} catch (error) {
    console.error('Error during login:', error);
}
};

// для смены языка с русского на английский и тд
const { t, i18n } = useTranslation();
// для перехода между страницами
const history = useHistory();
// при нажатии на кнопку регистрации
const handleRedirectToRegistration = () => {
    history.push('/registration');
};
// при нажатии на надпись Забыли пароль?
const handleRedirectToChangeCode = () => {
    history.push('/change-code');
};
const returnToLogin = () => {
    history.push('/login');
};
const returnOurTeam = () => {
    history.push('/ourteam');
};
const returnLinks = () => {
    history.push('/links');
};
// проверка корректны ли данные, которые введены в поля логина и пароля
const [showForgotPassword, setShowForgotPassword] = useState(false);
return (
    <div>
        {/** контейнер страницы */}
        <div className={style.LoginPage}>
            {/** заголовок*/}
            <div className={style.Header}>
                <a
                    onClick={returnToLogin}
                    className={style.HeaderLink}
                >

```

```

        Генератор Гена
    </a>
</div>
{/** основные компоненты */}
<div className="container-fluid">
    <div className={style.MainPage}>
        <div className="row">
            {/** пустота*/}
            <div className={'col-lg-2'}></div>
            {/** основная информация и ссылки для скачивания */}
            <div className={'col-lg-5 text-center'}>
                <div className={style.Info}>
                    <h2>Генератор Гена</h2>
                    <h3>Сервис для генерации изображения по промпту</h3>
                <div className={style.Link}>
                    <a
                        onClick={returnLinks}
                        className={style.text}
                    >
                        App Store
                    </a>
                    <a
                        onClick={returnLinks}
                        className={style.text}
                    >
                        Google Play
                    </a>
                    <a
                        onClick={returnLinks}
                        className={style.text}
                    >
                        Linux
                    </a>
                    <a
                        onClick={returnLinks}
                        className={style.text}
                    >
                        Windows
                    </a>
                </div>
            </div>
        </div>
        {/** форма*/}
        <div className={'col-lg-3'}>
            <div className={style.MainForm}>
                <form className={style.form} onSubmit={login} >
                    <h1>Войти</h1>
                    <MyInput
                        className={style.inputLogin}
                        type="text"
                        placeholder='Введите имя пользователя'
                    >

```

```

        onChange={handleUsernameChange}
      />
      <MyInput
        className={style.inputPass}
        type="password"
        placeholder='Введите пароль'
        onChange={handlePasswordChange}
      />
    { // условное отображение, чтобы можно было перейти на
      страницу смены пароля

      showForgotPassword
      ?
      <a
        onClick={handleRedirectToChangeCode}
        className={style.error}
      >
        Забыли пароль?
      </a>
      :
      <div className={style.Remember}>
        <p className={style.remember}>
          <input
            onChange={(e) =>
              setRemember(e.target.checked)}

            type="checkbox"
            checked name="remember"
          />
          Сохранить данные?
        </p>
      </div>
    }
    <MyButton

      type="submit"
      style={{ backgroundColor: '#1F5CB6', color: 'ffffff'

    }}

    >
      Войти
    </MyButton>
    <MyButton
      onClick={handleRedirectToRegistration}
      style={{ backgroundColor: 'ffffff', color: '#1F5CB6',

    margin: '0px' }}

    >
      Зарегистрироваться
    </MyButton>
  </form>
</div>
{/** QR */}
<div className={style.QR} >
  <div className={style.textQr}>

```

```
        <h5 className={style.text}>Да итоговый дизайн отличается от
дизайн макета</h5>

        <h6 className={style.text}>ИЗВИНИТЕ</h6>

        <a className={style.more}>фронтендер не умеет красить
кнопки</a>
```

```

        </div>
      </div>
      /** ссылки внизу страницы */
      <div className={style.Bottom}>
        <h4>Гена © 2025</h4>
        <a
          className={style.text}
          onClick={returnOurTeam}
          style={{ marginRight: "15%" }}
        >
          Github
        </a>
        <a
          onClick={() => i18n.changeLanguage('en')}
          className={style.text}
        >
          English
        </a>
        <a
          onClick={() => i18n.changeLanguage('ru')}
          className={style.text}
          style={{ marginRight: "10%" }}
        >
          Русский
        </a>
      </div>
    </div>
  </div>
</div>
</div>
</div>
);
};
export default LoginLight;

```

```
import React, { useContext, useState, Suspense, useEffect } from "react";
import { AuthContext } from "../context";
import style from "../styles/Light/Main.module.css";
import MyButton from "../UI/components/buttons/MyButton";
import MyInput from "../UI/components/input/MyInput";
import "bootstrap/dist/css/bootstrap.min.css";
import { useTranslation } from "react-i18next";
import { useHistory } from "react-router-dom";

const MainLight = () => {
```

```

const [isLoading, setIsLoading] = useState(false);
const [accessToken, setAccessToken] = useState(
  localStorage.getItem("accessToken")
);
const [requestData, setRequestData] = useState({ prompt: "" });
const handlePromptChange = (event) => {
  const prompt = event.target.value;
  setRequestData({ ...requestData, prompt });
};

const refreshAccessToken = async () => {
  try {
    const refreshToken = localStorage.getItem("refreshToken");
    console.log("refresh", refreshToken)
    if (!refreshToken) {
      throw new Error("Refresh token not found");
    }

    const response = await fetch("http://localhost:8000/api/users/refresh/", {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
        // Если сервер требует Authorization header (например, с accessToken)
        // Authorization: `Bearer ${localStorage.getItem("accessToken")}` ,
      },
      body: JSON.stringify({ refresh: refreshToken }), // или { refresh_token: refreshToken }
    });

    // Логирование для отладки
    console.log("Refresh token response status:", response.status);

    if (!response.ok) {
      const errorData = await response.json();
      console.error("Refresh token error details:", errorData);
      throw new Error("Failed to refresh token");
    }

    const data = await response.json();
    console.log("New tokens data:", data);

    // Проверка наличия access-токена в ответе
    if (!data.access) {
      throw new Error("Access token not found in response");
    }

    localStorage.setItem("accessToken", data.access);
    localStorage.setItem("refreshToken", data.refresh);
    setAccessToken(data.access);
    console.log("token", data.access)
    return data.access; // Возвращаем новый токен для использования
  } catch (err) {

```

```

        console.error("Error refreshing token:", err);
        // Дополнительные действия при ошибке:
        // - Удалить все токены
        // - Перенаправить на страницу входа
        localStorage.removeItem("accessToken");
        localStorage.removeItem("refreshToken");
        setAccessToken(null);
        window.location.href = "/login";
        throw err; // Пробросить ошибку дальше, если нужно
    }
};
// Запускаем таймер обновления токена при загрузке компонента
useEffect(() => {
    //refreshAccessToken();
    // Обновляем токен сразу при загрузке (по желанию)
    const intervalId = setInterval(() => {
        refreshAccessToken();
    }, 5 * 60 * 1000); // 5 минут

    return () => clearInterval(intervalId);
}, []);

const handleSubmit = async (event) => {
    event.preventDefault();
    setIsLoading(true);

    try {
        const userId = getUserIdFromToken();
        const token = localStorage.getItem("accessToken");
        console.log("meow")
        console.log("TOKEN: ", token)
        if (!token) {
            throw new Error("JWT token not found in localStorage");
        }

        const response = await fetch("http://localhost:8000/api/requests/", {
            method: "POST",
            headers: {
                "Content-Type": "application/json",
                Authorization: `Bearer ${token}`,
            },
            body: JSON.stringify({
                user_id: userId,
                prompt: requestData.prompt,
            }),
        });

        if (response.status === 201) {
            const data = await response.json();
            const operationId = data.operationID;

```



```

const fetchImageWithRetry = async (retries = 5, delay = 2000) => {
  for (let i = 0; i < retries; i++) {
    const imageResponse = await fetch(
      `http://localhost:8000/api/getimage/?operation_id=${operationId}&user_id=${userId}`,
      {
        headers: {
          Authorization: `Bearer ${token}`,
        },
      }
    );

    if (imageResponse.ok) {
      const blob = await imageResponse.blob();
      const imageUrl = URL.createObjectURL(blob);
      setImgSrc(imageUrl);
      return; // Успешно получили изображение, выходим из функции
    } else if (imageResponse.status === 404) {
      // Изображение ещё не готово, ждём и повторяем попытку
      await new Promise((res) => setTimeout(res, delay));
    } else {

      alert("Ошибка при получении изображения");
      return;
    }
  }
  alert("Изображение не появилось в течение ожидания");
};

await fetchImageWithRetry();
} else {
  alert(`Ошибка при отправке запроса: статус ${response.status}`);
}
} catch (error) {
  alert(error.message);
} finally {
  setIsLoading(false);
}
};

const [imgSrc, setImgSrc] = useState(null);
function getUserIdFromToken() {
  const token = localStorage.getItem("accessToken"); // Получаем токен из localStorage
  if (!token) {
    throw new Error("JWT token not found in localStorage");
  }

  // Декодируем токен (предполагаем, что он закодирован в формате base64)
  const payload = token.split(".")[1]; // Берем среднюю часть токена
  const decodedPayload = JSON.parse(atob(payload)); // Декодируем base64 и парсим JSON
  console.log(token);
  return decodedPayload.user_id; // Предполагаем, что ID пользователя хранится в поле userId
}

```

```

// Функция для отправки запроса на историю запросов
const [history, setHistory] = useState([]);
const [showHistory, setShowHistory] = useState(false); // Состояние для отображения истории
const [selectedRequest, setSelectedRequest] = useState(null); // Состояние для хранения информации
о выбранном запросе

```

```

const fetchUserHistory = async () => {
  try {
    setIsLoading(true); // Начинаем загрузку
    const userId = getUserIdFromToken();
    const token = localStorage.getItem("accessToken");
    if (!token) {
      throw new Error("Access token not found");
    }

    const response = await fetch(
      `http://localhost:8000/api/users/history/?user_id=${userId}`,
      {
        method: "GET",
        headers: {
          "Content-Type": "application/json",
          Authorization: `Bearer ${token}`, // формат токена для JWT
        },
      }
    );

    if (!response.ok) {
      throw new Error(`Ошибка при получении истории: ${response.status}`);
    }

    const data = await response.json();
    console.log(data[2].operationID)
    // Предполагаем, что data – массив запросов, сортируем и берём последние 3
    const lastThree = data.slice(-3).reverse();
    setHistory(lastThree);
    setShowHistory(true); // Показываем историю после загрузки
  } catch (err) {
    alert(err.message);
  } finally {
    setIsLoading(false); // Заканчиваем загрузку
  }
};

```

```

// Функция для получения информации о конкретном запросе
const fetchRequestDetails = async (requestId) => {
  try {
    setIsLoading(true);
    console.log(requestId)
    const userId = getUserIdFromToken();
    const token = localStorage.getItem("accessToken");

```

```

const url = new URL(`https://your-api-domain.com/api/requests/${requestId}`);
url.searchParams.append('user_id', userId);

if (!token) {
  throw new Error("Access token not found");
}

const response = await fetch(
  url.toString(),
  {
    method: "GET",
    headers: {
      "Content-Type": "application/json",
      Authorization: `Bearer ${token}`,
    },
  }
);

if (!response.ok) {
  throw new Error(`Ошибка при получении деталей запроса: ${response.status}`);
}

const data = await response.json();
setSelectedRequest(data);
} catch (err) {
  alert(err.message);
} finally {
  setIsLoading(false);
}
};

const getItemStyle = (item) => {
  return {
    backgroundColor: item.success ? "#e6ffe6" : "#ffe6e6",
    margin: "5px 0",
    borderRadius: "5px",
    cursor: "pointer",
  };
};

async function deleteUser() {
  const userId = getUserIdFromToken();
  const token = localStorage.getItem("accessToken");
  try {
    const response = await fetch(`http://localhost:8000/api/users/?user_id=${userId}`, {
      method: 'DELETE',
      headers: {
        'Content-Type': 'application/json',
        'Authorization': `Bearer ${accessToken}`
      }
    })
  }

```

```

    });

    if (!response.ok) {
      throw new Error(`Ошибка HTTP: ${response.status}`);
    }

    const data = await response.json();
    console.log('Пользователь удалён:', data);
  } catch (error) {
    console.error('Ошибка при удалении пользователя:', error);
  }
}

// переход между страницами
const historyHook = useHistory();
const returnToLogin = () => {
  historyHook.push("/login");
};
const returnToChangePass = () => {
  historyHook.push("/change-pass");
};
const returnOurTeam = () => {
  historyHook.push("/ourteam");
};
const { t, i18n } = useTranslation("translation");
return (
  <div>
    <Suspense fallback={<div>Loading...</div>}></Suspense>
    /** контейнер страницы */
    <div className={style.LoginPage}>
      /** заголовок*/
      <div className={style.Header}>
        <a onClick={returnToLogin} className={style.HeaderLink}>
          Генератор Гена
        </a>
      </div>
      <div className="container-fluid">
        <div className={style.MainPage}>
          <div className="row">
            <div className="col-lg-2"></div>
            <div className={`col-lg-5 text-center`}>
              <div className={style.Info}>
                <h2>Генератор Гена</h2>
              </div>
              <div style={{ height: "80% " }}>
                {imageSrc && (
                  <img src={imageSrc} alt="Полученное изображение" />
                )}
              </div>
            </div>
            <div className={`col-lg-3`}>
              <div className={style.MainForm}>

```

```

<form className={style.form}>
  <h1>Введите промпт</h1>
  <MyInput
    style={{ margin: "0px", marginTop: "8%" }}
    type="text"
    placeholder="Введите промпт"
    onChange={handlePromptChange}
  />
  <MyButton
    style={{
      backgroundColor: "#1F5CB6",
      color: "ffffff",
      margin: "0px",
      marginTop: "8%",
    }}
    onClick={handleSubmit}
  >
    Продолжить
  </MyButton>
  <MyButton
    style={{
      backgroundColor: "#1F5CB6",
      color: "ffffff",
      margin: "0px",
      marginTop: "8%",
    }}
    onClick={fetchUserHistory}
    disabled={isLoading}
  >
    История
  </MyButton>
</form>
{showHistory && (
  <div>
    <h2>История запросов</h2>
    <ul>
      {history.map((item, index) => (
        <li
          key={index}
          style={getItemStyle(item)}
          onClick={() => fetchRequestDetails(item.operationID)} // Обработчик
        >
          {item.prompt || JSON.stringify(item)}
        </li>
      ))}
    </ul>
  </div>
)}

{selectedRequest && (

```

клика

```

<div>
  <h2>Детали запроса</h2>
  <p>ID: {selectedRequest.id}</p>
  <p>Prompt: {selectedRequest.prompt}</p>
  {selectedRequest.image && (
    <img
      src={selectedRequest.image}
      alt="Изображение запроса"
    />
  )}
</div>
)}
</div>
<div className={style.QR}>
  <MyButton
    onClick={deleteUser}
    style={{ backgroundColor: 'ffffff', color: '#1F5CB6', margin: '15px' }}
  >
    Удалить пользователя
  </MyButton>
  <MyButton
    onClick={returnToChangePass}
    style={{ backgroundColor: 'ffffff', color: '#1F5CB6', margin: '15px' }}
  >
    Поменять пароль
  </MyButton>
</div>
</div>
{/** ссылки внизу страницы */}
<div className={style.Bottom}>
  <h4>Гена © 2025</h4>
  <a
    className={style.text}
    onClick={returnOurTeam}
    style={{ marginRight: "15%" }}
  >
    Github
  </a>
  <a
    onClick={() => i18n.changeLanguage("en")}
    className={style.text}
  >
    English
  </a>
  <a
    onClick={() => i18n.changeLanguage("ru")}
    className={style.text}
    style={{ marginRight: "10%" }}
  >
    Русский
  </a>

```

```

        </div>
      </div>
    </div>
  </div>
</div>
</div>
);
};
export default MainLight;

import React, { useContext, useState } from "react";
import { AuthContext } from "../context";
import style from "../styles/Light/Registration.module.css"
import MyButton from "../UI/components/buttons/MyButton";
import MyInput from "../UI/components/input/MyInput";
import 'bootstrap/dist/css/bootstrap.min.css';
import { useTranslation } from 'react-i18next';
import { useHistory } from 'react-router-dom';

const RegLight = () => {
  const { isAuth, setIsAuth } = useContext(AuthContext);
  const [isLoading, setIsLoading] = useState(false);
  const [error, setError] = useState(null);
  const [success, setSuccess] = useState(false);

  const [loginData, setLoginData] = useState({ username: '', password: '', email: '' });
  const handleUsernameChange = (event) => {
    const username = event.target.value;
    setLoginData({ ...loginData, username });
  };

  const handlePasswordChange = (event) => {
    const password = event.target.value;
    setLoginData({ ...loginData, password });
  };

  const handleEmailChange = (event) => {
    const email = event.target.value;
    setLoginData({ ...loginData, email });
  };

  const handleSubmit = async (event) => {
    event.preventDefault();
    setIsLoading(true);
    setError(null);

    try {
      const response = await fetch('http://localhost:8000/api/users/register/', {
        method: 'POST',
        headers: {
          'Content-Type': 'application/json',
        },

```

```

        body: JSON.stringify(loginData)
    });
    console.log('Registration successful:');
    if (!response.ok) {
        const errorData = await response.json();
        throw new Error(errorData.message || 'Registration failed');
    }

    const data = await response.json();
    console.log('Registration successful:', data);
    setSuccess(true);
    handleRedirectToLogin();

    } catch (err) {
        console.error('Registration error:', err);
        setError(err.message || 'Something went wrong');
    } finally {
        setIsLoading(false);
    }
};

// переход между страницами
const history = useHistory();
const handleRedirectToLogin = () => {
    history.push('/login');
};
const returnToLogin = () => {
    history.push('/login');
};
const returnOurTeam = () => {
    history.push('/ourteam');
};
const { t, i18n } = useTranslation('translation');
return (
    <div>
        {/** контейнер страницы */}
        <div className={style.LoginPage}>
            {/** заголовок*/}
            <div className={style.Header}>
                <a
                    onClick={returnToLogin}
                    className={style.HeaderLink}
                >
                    Генератор Гена
                </a>
            </div>
            <div className="container-fluid">
                <div className={style.MainPage}>
                    <div className="row">
                        <div className={'col-lg-2'}></div>
                        <div className={'col-lg-5 text-center'}>
                            <div className={style.Info}>

```



```

<h2>Генератор Гена</h2>
<h3 className={style.text}>Сервис для генерации изображения по
промпту</h3>

<div className={style.Link}>
  <a className={style.text}>App Store</a>
  <a className={style.text}>Google Play</a>
  <a className={style.text}>Linux</a>
  <a className={style.text}>Windows</a>
</div>
</div>
</div>
<div className={`col-lg-3`} >
  <div className={style.MainForm}>
    <form className={style.form} >
      <h1>Начать пользоваться</h1>
      <MyInput
        style={{ margin: '0px', marginTop: '8%' }}
        type="text"
        placeholder='Введите имя пользователя'
        onChange={handleUsernameChange}
      />
      <MyInput
        style={{ margin: '0px', marginTop: '8%' }}
        type="text"
        placeholder='Введите email'
        onChange={handleEmailChange}
      />
      <MyInput
        style={{ margin: '0px', marginTop: '8%' }}
        type="password"
        placeholder='Введите пароль'
        onChange={handlePasswordChange}
      />
      <MyButton
        style={{ backgroundColor: '#1F5CB6', color: 'ffffff',
margin: '0px', marginTop: '8%' }}

        onClick={handleSubmit}
      >
        Продолжить
      </MyButton>
    </form>
  </div>
  <div className={style.QR} >
    <div className={style.textQR}>
      <h5 className={style.text}>Да итоговый дизайн отличается от
дизайн макета</h5>

      <h6 className={style.text}>ИЗВИНИТЕ</h6>
      <a className={style.more}>Фронтендер не умеет красить
кнопки</a>
    </div>
  </div>
</div>

```

```

    </div>
    /** ссылки внизу страницы */
    <div className={style.Bottom}>
      <h4>Гена © 2025</h4>
      <a
        className={style.text}
        onClick={returnOurTeam}
        style={{ marginRight: "15%" }}
      >
        Github
      </a>
      <a
        onClick={() => i18n.changeLanguage('en')}
        className={style.text}
      >
        English
      </a>
      <a
        onClick={() => i18n.changeLanguage('ru')}
        className={style.text}
        style={{ marginRight: "10%" }}
      >
        Русский
      </a>
    </div>
  </div>
</div>
</div>
</div>
</div>
);
};
export default RegLight;

```

Маршруты:

```

//import ChangeCode from "../pages/ChangeCode.jsx";
import ChangePass from "../pages/ChangePass.jsx";
//import CodeLight from "../pages/Light/CodeLight.jsx";
//import Links from "../pages/Links.jsx";
import Login from "../pages/Login.jsx";
import Main from "../pages/Main.jsx";
//import OurTeam from "../pages/OurTeam.jsx";
import Registration from "../pages/Registration.jsx";

export const privateRoutes = [
]

export const publicRoutes = [
  {path: '/login', component: Login, exact: true},
  {path: '/registration', component: Registration, exact: true},
  //{path: '/code', component: CodeLight, exact: true},

```

```
{path: '/main', component: Main, exact: true},  
  //{path: '/change-code', component: ChangeCode, exact: true},  
  {path: '/change-pass', component: ChangePass, exact: true},  
  //{path: '/ourteam', component: OurTeam, exact: true},  
  // {path: '/links', component: Links, exact: true}  
]
```

ПРИЛОЖЕНИЕ Г

В приложении приведен docker compose.

```
version: "3.9"

services:
  app:
    build:
      context: .
      dockerfile: Dockerfile
    environment:
      DJANGO_DB_HOST: db
      DJANGO_DB_NAME: postgres_tsp_db
      DJANGO_DB_USER: postgres
      DJANGO_DB_PASS: postgres
      DJANGO_DB_PORT: "5432"
      DJANGO_DEBUG: "False"
    depends_on:
      db:
        condition: service_healthy
      ml_service: # <-- Зависимость от ML-сервиса
        condition: service_started
    ports:
      - "8000:8000"

  frontend:
    build:
      context: ./frontend
      dockerfile: Dockerfile
    ports:
      - "3000:3000"
    environment:
      - REACT_APP_API_URL=http://app:8000
    depends_on:
      - app

  ml_service:
    build:
      context: ./ml_service # путь к директории с нейросетью
      dockerfile: Dockerfile
    ports:
      - "5000:5000"
    restart: unless-stopped

  db:
    image: postgres:latest
    environment:
      POSTGRES_DB: postgres_tsp_db
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
    ports:
```

```
- "5432:5432"
volumes:
  - db_data:/var/lib/postgresql/data
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U postgres -d postgres_tsp_db"]
  interval: 10s
  timeout: 5s
  retries: 5

volumes:
  db_data:
```